

Machine Learning-Based Anomaly Detection in Digital System Monitoring

Technical documentation

Bachelor's project by:

Anri Sitchinava & Tinatin Tavadze

Introduction

The idea for this project was born from the real struggles we faced in our work every day. As network operations center engineers, we have to stay organized and be able to recognize/handle production incidents as soon as possible. However, too much noise makes this difficult. The problem is traditional monitoring systems, which use threshold-based alerts and fail to recognize expected changes in behavior (like increased memory/CPU usage due to scheduled events/high activity hours).

The project aims to fix the problem by creating smarter monitoring system using machine learning. This is achieved by training different models on real data and evaluating their performance. We determined what works best in our scenario and managed to reduce false alerts as a result.

Repositories

Notebooks (under notebooks/)

All notebooks read data/labeled.csv. The data is split and loaded 80/20, trained, detections are plotted and results are evaluated.

- **isolation_forest_model.ipynb** – This is the starting notebook. To begin with, it shows one-week zoomed data to make recurring trend more visible. The first step of our project was comparing static threshold based approach to Isolation forest. Notebook contains both approaches plot detection and reports their precision, recall, F1 score and confusion matrices. We also have statsmodels logistic-regression analysis.

- **lof_model.ipynb** – Local Outlier Factor (LOF), a density-based anomaly detection. It flags points if local neighborhood density is much lower than their neighbors. It's particularly strong in detecting sudden spikes and drops.
- **ocsvm_model.ipynb** – One-Class SVM, a boundary-based anomaly detection. It learns what is the boundary of normal data and flags new points outside this boundary as anomalies. The notebook Includes the StandardScaler step that the RBF kernel requires.
- **ensemble_model.ipynb** – A high-precision ensemble that flags a point only when both LOF and Isolation Forest flag it as anomaly (logical AND). From the simple models, this has the fewest false positives.
- **best_unsupervised.ipynb** – This is the best unsupervised learning approach we developed. Instead of relying on only one label-free model, we take averages of normalized scores of 4 models (LOF, two Isolation forests, PCA reconstruction error). Then we apply strict/loose thresholds to confirm anomalies with neighbors and fill the gaps.
- **random_forest_model.ipynb** – A supervised Random Forest. It trains on labeled data and produces the best results of all models. Shows a feature-importance bar chart.
- **final_comparison.ipynb** – This notebook is a summary of the project. It contains one week trend, all six models results, chart of precision/recall/F1, confusion matrices, a best-unsupervised-vs-best-supervised detection plot, and project's main findings.

Data (under data/)

- **memory_logs_raw.xlsx** – Data is taken from a production VM - 16,992 rows (Feb 1 - Mar 31, 2025), 5 minute intervals. Columns contain Timestamp, Memory_Usage_Pct, Is_Anomaly, Notes. Average memory usage is ~41%, standard deviation - 4.4%. Data

has known recurring pattern - memory spikes during US market open at 13:30–13:45 GMT on weekdays. It's used for data preparation and turned into labeled.csv.

- **labeled.csv** – This is the same 16,992 rows of data ready to be read and trained by models. This one has 22 columns: the timestamp, memory_pct, the anomaly label similar to the original export, and 19 engineered features. Every model is trained on this.

Models (under models/)

- **isolation_forest.pkl** – A saved copy of an already trained Isolation forest model. This means that we can reuse the model without having to train it again in the future.
- **scaler.pkl** – The fitted StandardScaler used to standardize features for the scale-sensitive models (One-Class SVM, PCA). It has to be applied to any new data before prediction.

System Architecture and Design

The system fully runs locally inside Jupyter kernel without server or database. Raw exported data first goes through feature engineering, it's injected with synthetic anomalies, is converted to model-ready dataset. The dataset goes through anomaly detection models, and finally the results are visualized and evaluated. This keeps the project reproducible and transparent.

The architecture is in classic batch ML pipeline, where each stage's output is the next input. Monitoring data is analyzed in bulk after export. Each stage (notebook) can be run/re-run independently and new model can be added separately without impacting the others. Each model is trained on the same data (labeled.csv file) which makes it easier to compare them to each other.

Injected anomalies

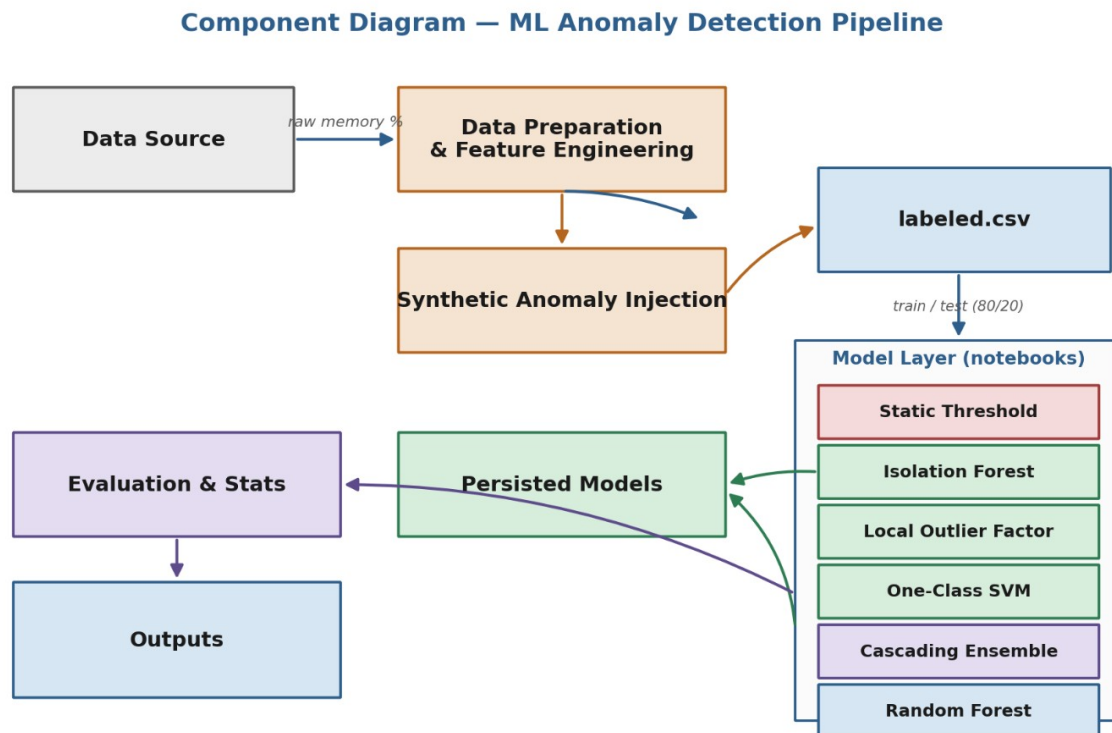
We inject 5 anomaly types for evaluation (250 anomaly points, 1.47% of dataset):

- **5 short spikes (single-point jumps to 85–95%)**

- 5 short drops (single-point drops to 15–25%)
- 4 prolonged increases (2-hour shifts +20–30%)
- 4 prolonged drops (1-hour shifts to 20–28%)
- 2 gradual ramps (slow rise +35% over 4 hours)

Component diagram

ML anomaly detection pipeline visualized



Data flow diagram

Data-Flow Pipeline



Features

After feature engineering, we ended up with 22 features for our data. Statsmodels logistic regression gives us information about which of them is significant by calculating the p value.

Statistically significant features ($p < 0.05$)

- **memory_pct** ($p = 2e-8$) – Memory usage at a given time.
- **roll_std_1h** ($p = 2e-6$) – How much memory has been jumping in the last hour (standard deviation).
- **dev_from_hour** ($p = 3e-7$) – How deviated value is for that time.
- **roll_mean_24h** ($p = 0.001$) – Average memory level over the last 24 hours.
- **zscore_1h, zscore_hour, zscore_24h** (all $p < 0.05$) – These measure how unusual value is compared to last hour, this hour of day, or last 24 hours.
- **diff_24** ($p < 0.05$) – how much has memory changed in from 24 reading ago (approximately 2 hours).

Not statistically significant

- **diff_1, diff_6, diff_48, diff_96** (correlated with each other)

- **day_of_week, is_weekend** (less important than time-of-day)
- **hour, minute, time_of_day** (NaN — multicollinear with each other)

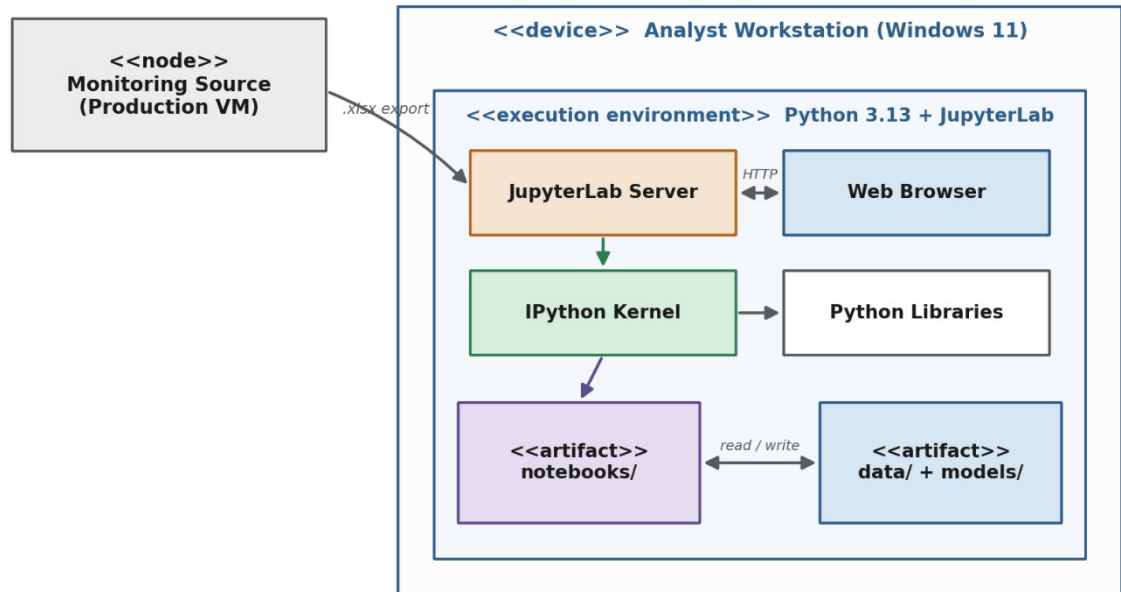
Models selected

We chose to implement 6 approaches, including traditional static threshold approach, supervised learning and unsupervised learning.

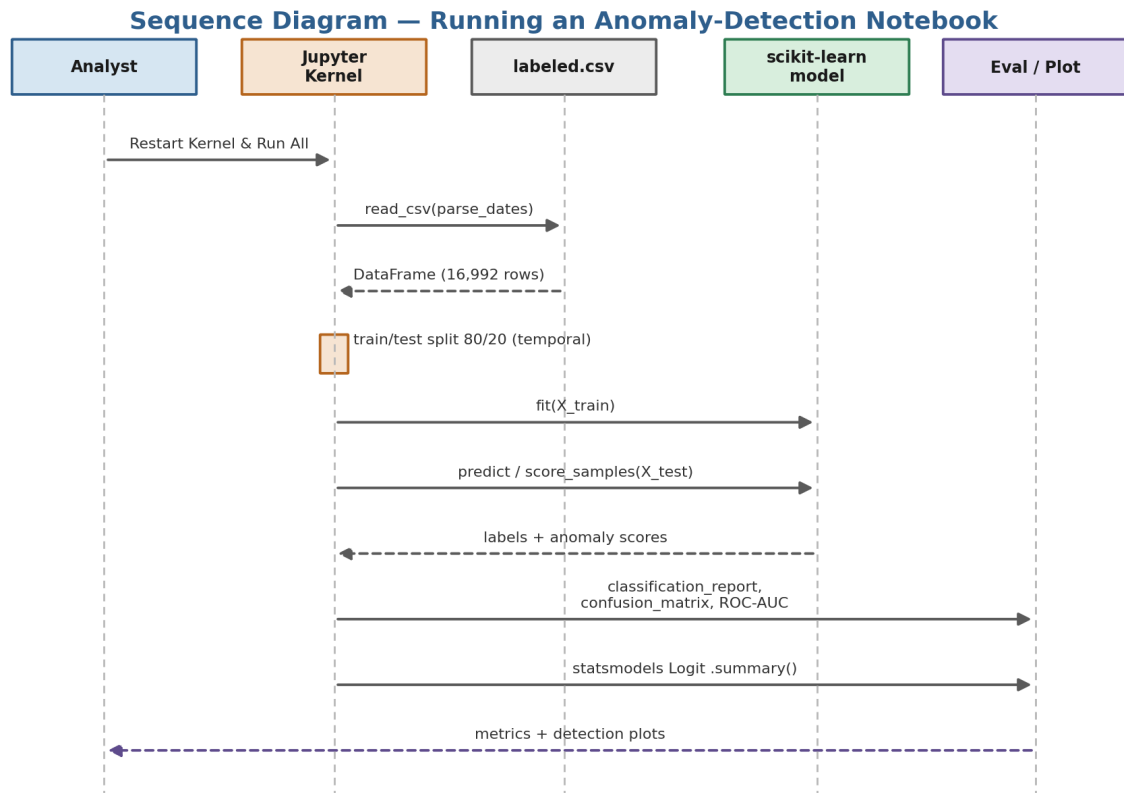
- **Static threshold** – This is a traditional approach that we want to find an improvement for. This is what motivated the project.
- **Isolation forest** – Unsupervised learning which is fast and easy to implement. It isolates outliers and random splits.
- **Local outlier factor** – Best unsupervised learning anomaly detection we tried. Compares local density to neighbors.
- **One-class SVM** – Unsupervised learning which learns boundary around normal data.
- **Cascading ensemble** – Averages four anomaly detection models, best unsupervised learning.
- **Random forest** – Supervised learning trained on labels. The best results overall, but not usable for new unlabeled data.

Deployment

Deployment Diagram — Analyst Workstation



Sequence diagram



Technologies

- **Language** – Python 3.13.
- **Environment** – JupyterLab.
- **Data handling** – pandas, NumPy, openpyxl.
- **Machine learning** – scikit-learn.
- **Statistics** – statsmodels.
- **Visualization** – matplotlib, seaborn.
- **Persistence** – joblib.

Installation and configuration

The project can be easily run by any machine with Python 3 and required packages.

Prerequisites

- **Python 3.10 or newer** – Verify with command: `python --version`.
- **Pip** – Verify with command: `pip --version`.
- **Around 200MB free disk** – For environment and files.
- **A web browser (Chrome, Firefox, Microsoft edge)** – To run JupyterLab.

Installation

- **Run git clone and cd to created folder:**

```
git clone https://github.com/AnriSichinaval/anomaly-  
detection.git  
cd anomaly-detection
```

- **Create virtual environment.**

Windows (PowerShell):

```
python -m venv .venv  
.venv\Scripts\Activate.ps1
```

MacOS/Linux:

```
python3 -m venv .venv  
source .venv/bin/activate
```

- **Install dependencies:**

```
pip install pandas numpy openpyxl scikit-learn statsmodels
```

```
matplotlib seaborn joblib jupyterlab
```

- **Launch JupyterLab from project root:**

```
jupyter lab
```

- **Open a notebook** – from notebooks/ folder (starting from isolation_forest_model.ipynb)

- **Run the notebook** – from the menu choose: Kernel – Restart Kernel and Run All Cells.

It will take under a minute.

Dependencies and versions

- **Python** – 3.10 (3.13 used).
- **pandas** – 2.0+.
- **numpy** – 1.24+.
- **openpyxl** – 3.1+.
- **scikit-learn** – 1.3+.
- **statsmodels** – 0.14+.
- **matplotlib** – 3.7+.
- **seaborn** – 0.12+.
- **joblib** – 1.3+.
- **jupyterlab** – 4.0+.

Troubleshooting guide

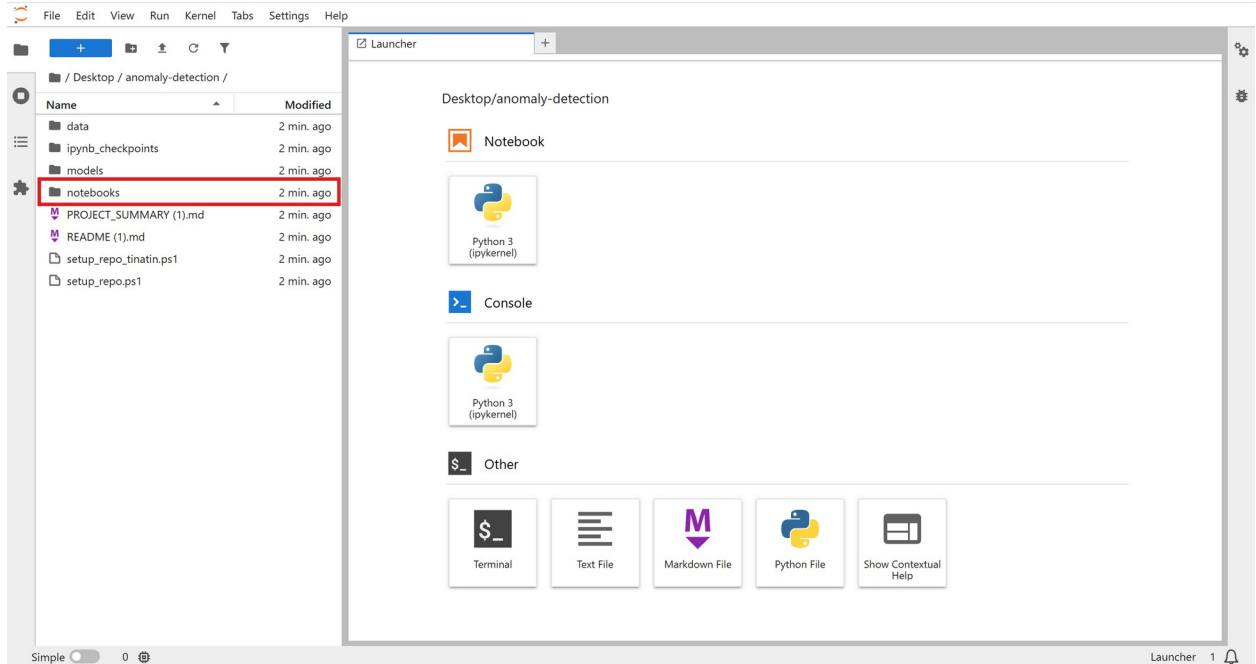
- **Plots not appearing** – Make sure matplotlib is installed, run the cells with Restart Kernel and Run All.
- **Kernel is slow or hangs on One-Class SVM** – SVM scales poorly with many rows so this is expected, let it finish the job.
- **FileNotFoundError:../data/labeled.csv** – JupyterLab was not launched from the project root, close it and relaunch from the root folder.
- **The browser tab never opens after running jupyter lab** – Copy the `http://localhost:8888` link shown in the terminal and paste it into your browser yourself.
- **Port 8888 is already in use** – A JupyterLab session is already running, close the other terminal/tab and rerun the command.

User manual

Starting JupyterLab

1. Open a terminal in the project root.
2. Type `jupyter lab` and enter, the browser will open.

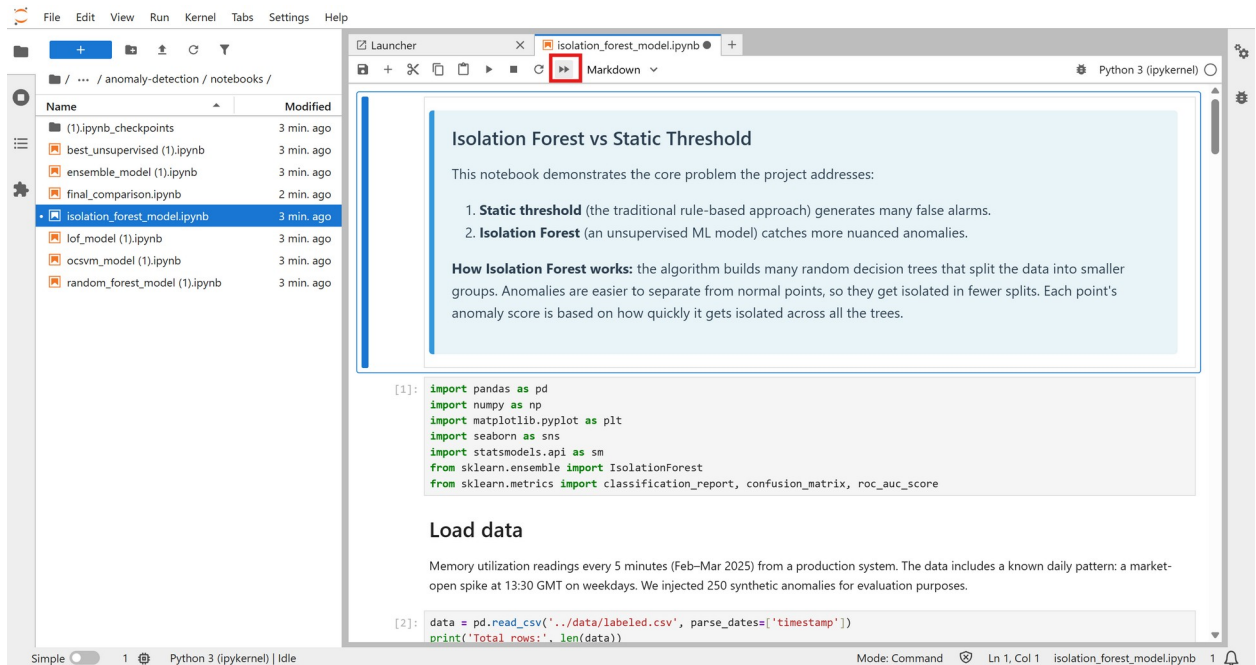
3. In the left file browser, open notebooks/ folder (highlighted with a red box).



Running a notebook

1. Double click, for example, `isolation_forest_model.ipynb` to open it.

- From the top menu choose 'Kernel – Restart Kernel and Run All Cells' and confirm (highlighted with a red box).



- Cells will execute in a few seconds.

Reading the output

- Dataset summary** – Total rows, number of anomalies, how many fall in the test set.
- Detection plot(s)** – Memory over time in light blue and color coded points:
 - Green** – Anomalies the model flagged.
 - Red X** - Anomalies the model missed.
 - Orange triangle** – False alarms.
- Classification report** – Precision, recall, and F1 for normal and anomaly classes.
- Confusion matrix** – Counts false/true positives and negatives.
- ROC-AUC** – A threshold-independent quality score (close to 1 – good).

- **Statistical summary** (main and random forest notebooks) – the statsmodels logistic-regression table.

Conclusion

This project shows that machine learning can improve traditional static-threshold monitoring, which raises too many false alarms on expected behavior. Using two months of real memory data with injected anomalies, we compared a static threshold against a different anomaly detection models and found a clear result: the threshold flags every spike as a false alarm, while the machine-learning models learn what pattern is normal and focus on true anomalous behavior.

The supervised Random Forest set the performance ceiling at $F1 = 0.83$ but needs labelled data, which is unrealistic to have for every single machine. The cascading unsupervised ensemble reached $F1 = 0.74$ using no labels, which makes it practical to deploy across many new machines with no labelling. Both approaches can be used to replace static threshold based monitoring – random forest for important machines and cascading ensemble for new unlabeled VMs.

In conclusion, we demonstrated that machine-learning based anomaly detection can be used to have smarter monitoring. We provided reproducible pipeline which makes it possible to use with new machines in the future.